Problem 1: Tree Terminology Identification

Given Tree:

```
      CEO
     /  \
    CTO   CFO
   / \    \
Dev Lead HR
  /  \
 Dev1  Dev2
```

a) Leaf Nodes

Leaf = node with **no children**

➡️ **Leaf nodes:**
Dev1, Dev2, HR

---

b) Height of the Tree

Height = number of **edges in longest path from root to leaf**

Longest path:
CEO → CTO → Dev Lead → Dev1 → **3 edges**

➡️ **Height = 3**

---

c) Depth of "Dev Lead"

Depth = number of edges from root to that node

CEO → CTO → Dev Lead → **2 edges**

➡️ **Depth = 2**

---

d) Ancestors of "Dev1"

Ancestors = all nodes from root to parent

Path:
CEO → CTO → Dev Lead → Dev1

➡️ **Ancestors: CEO, CTO, Dev Lead**

---

e) Degree of CTO

Degree = number of children

CTO has: Dev Lead, HR → **2 children**

➡️ **Degree = 2**

Problem 2: Traversal Application

Given Tree:

```
      root
     /   \
    home   var
   /  \     \
 user  docs   log
  /
config
```

---

a) Alphabetical listing (if BST)

➡️ Use **Inorder Traversal**
Reason: In BST → inorder = **sorted order**

---

b) Total size of directory

➡️ Use **Postorder Traversal**
Reason: process children first, then parent (bottom-up sum)

---

c) Backup entire structure

➡️ Use **Preorder Traversal**
Reason: copy parent before children (top-down replication)

---

d) Traversals

Inorder (LNR)

user subtree → home → docs → root → var → log

➡️ **config, user, home, docs, root, var, log**

Preorder (NLR)

➡️ **root, home, user, config, docs, var, log**

Postorder (LRN)

➡️ **config, user, docs, home, log, var, root**

---

e) Why Postorder for deletion?

Because:

- Children are deleted **before parent**
- Prevents dangling references

➡️ Safe bottom-up deletion

Problem 3: BST Construction and Validation

Insert order:

50, 30, 70, 20, 40, 60, 80, 10, 25

---

a) Resulting BST

```
      50
    /   \
   30     70
  / \   / \
 20  40 60 80
/ \
10 25
```

---

b) Search for 25 (comparisons)

1. 25 < 50 → go left
2. 25 < 30 → go left
3. 25 > 20 → go right
4. Found 25

➡️ **Comparisons: 50 → 30 → 20 → 25**

---

c) Inorder Traversal

➡️ **10, 20, 25, 30, 40, 50, 60, 70, 80**

✔️ Sorted → Valid BST

---

d) Is this a valid BST?

```
     50
    /   \
   30     70
  / \   / \
 20  65 60 80
```

❌ **Invalid**

Reason:

- 65 is in left subtree of 50 but **> 50**
- Violates BST rule

---

e) Height of skewed BST (9 nodes)

Completely skewed → like linked list

➡️ Height = **8**

Balanced tree height ≈ **3**

➡️ Huge difference → shows importance of balancing

Problem 4: BST Operations

Initial BST:

```
      15
    /   \
   10     20
  / \   / \
 8   12 17  25
```

---

a) Delete node 10 (2 children)

Step:

- Replace with **inorder successor (12)**

Result:

```
      15
    /   \
   12     20
  /     / \
 8     17  25
```

---

b) Insert 14

14 < 15 → left
14 > 12 → right

```
      15
    /   \
   12     20
  / \   / \
 8   14 17  25
```

---

c) Insert 9

9 < 15 → left
9 < 12 → left
9 > 8 → right

```
      15
    /   \
   12     20
  / \   / \
 8   14 17  25
```

```
 \
  9
```

---

d) Range [10, 20]

➡️ Use **Inorder Traversal with pruning**

Why:

- BST property allows skipping irrelevant subtrees
- Efficient range queries

---

e) Time Complexity (search 25)

- **Best case:** O(1) (if root)
- **Worst case:** O(n) (skewed tree)

Balanced case → **O(log n)**

Problem 5: Expression Tree

Given:

```
    ×
   / \
  +   -
 / \ / \
3 5 8  2
```

---

a) Postorder (Postfix)

➡️ **3 5 + 8 2 - ×**

✔️ Operators come **after operands**

---

b) Inorder (Infix)

➡️ **(3 + 5) × (8 - 2)**

✔️ Add parentheses to preserve precedence

---

c) Preorder (Prefix)

➡️ **× + 3 5 - 8 2**

✔️ Operator comes first

---

d) Evaluation using Postorder (Stack)

Algorithm:

1. Scan left → right
2. If operand → push
3. If operator:
    - pop 2 values
    - apply operator
    - push result

Example:

3 5 + → 8
8 2 - → 6
8 × 6 → 48

➡️ **Final Answer = 48**

---

e) Expression Tree: a × b + c / d - e

Interpretation:

((a × b) + (c / d)) - e

Tree:

```
     -
    / \
   +   e
  / \
 ×   /
/ \ / \
a b c d
```

Traversals:

Inorder

➡️ **((a × b) + (c / d)) - e**

Preorder

➡️ **- + × a b / c d e**

Postorder

➡️ **a b × c d / + e -**